

Verificación Computacional: Rango y Singularidad 2R

Este script refuerza conceptualmente la relación entre la geometría del robot, la evaluación numérica del Jacobiano y el cálculo algorítmico del rango matricial y su determinante[cite: 12].

```
import numpy as np

def jacobian_2r(q1, q2, a1=1.0, a2=1.0):
    s1 = np.sin(q1)
    c1 = np.cos(q1)
    s12 = np.sin(q1 + q2)
    c12 = np.cos(q1 + q2)

    return np.array([
        [-a1*s1 - a2*s12, -a2*s12],
        [ a1*c1 + a2*c12,  a2*c12]
    ], dtype=float)

q_regular = (0.0, np.pi/2)
q_singular = (0.0, 0.0)

J_regular = jacobian_2r(*q_regular)
J_singular = jacobian_2r(*q_singular)

for name, J in [("regular", J_regular), ("singular",
    J_singular)]:
    print(f"\n{name}")
    print(np.round(J, 4))
    print("rank =", np.linalg.matrix_rank(J))
    print("det =", np.linalg.det(J))
```

Interpretación Física de la Salida[cite: 12]

Caso Regular: Muestra Rango = 2 y Determinante = 1. Posee dos direcciones instantáneas de velocidad planar independientes.

Caso Singular: Muestra Rango = 1 y Determinante = 0. Ha perdido una dirección de velocidad instantánea independiente en el espacio de tareas. El uso del determinante aquí es matemáticamente válido únicamente porque el sub-Jacobiano de traslación extraído es cuadrado (2×2).